BROTAR

Guía de demostración y estudio
Sprint 2 - módulo de campañas

Equipo: Ricardo, Alison y Santiago. Corte: 5 de octubre de 2026. Base: incremento integrado local en RicardoDev; frontend React/TypeScript, backend NestJS/TypeORM y PostgreSQL V2.

Propósito y lectura

Explicar qué debía cumplir cada tarea, dónde está implementada y cómo demostrarla. Primero preparar el entorno; después realizar el recorrido completo y consultar las trece fichas. Cada ficha incluye los criterios del backlog y un ensayo negativo. Las rutas de código son relativas a la raíz del proyecto y las líneas se verifican al generar este PDF.

Estado real del corte

Trece tareas con verificación técnica local. S2-21 y el criterio externo común conservan pendiente el commit/entrega y aceptación de líderes. No se ha publicado este corte ni modificado DEV, main o Trello. “Cierre técnico” no significa cerrar toda una historia BG del MVP.

Alcance funcional

Crear y guardar campañas propias; plan, presupuesto, financiamiento y recompensas; revisión final, envío, decisión administrativa, corrección, aprobación, publicación y catálogo público real. Se mantiene la base de acceso, perfiles y organizaciones del Sprint 1.

Fuera del alcance

Aportes, pagos, checkout, desembolsos, reembolsos, conciliación, KYC/KYB y rendición. No se añaden reglas comerciales definitivas pendientes del cliente. ADMIN publica de forma provisional conforme a D05; D06 sigue pendiente.

Resultado automatizado: 276 pruebas aprobadas, sin fallos ni omisiones. No constituye una auditoría de seguridad integral ni garantía de ausencia de errores.

01. Mapa de tareas y alcance

El identificador se conserva aunque tenga saltos: tareas trasladadas al Sprint 3 no se renumeran. S2-16 es una tarea compuesta vinculada a varias historias; no son seis tareas independientes.

Tarea	Responsable	Resultado del incremento
S2-02	Santiago	Planificar actividades e hitos
S2-03	Santiago	Registrar presupuesto y riesgos
S2-04	Santiago	Configurar el financiamiento
S2-05	Santiago	Configurar recompensas y preventas
S2-10	Santiago	Revisar la campaña antes del envío
S2-11	Santiago	Enviar la campaña a revisión
S2-12	Ricardo	Decidir sobre campañas recibidas
S2-13	Santiago	Corregir observaciones y reenviar
S2-14	Ricardo	Publicar y controlar estados de campaña
S2-15	Ricardo	Gestionar proyectos propios y descartar borradores
S2-16	Alison	Conectar catálogo y detalle público con campañas reales
S2-17	Ricardo	Preparar acceso y cola administrativa de campañas
S2-21	Ricardo	Verificar y documentar el segundo incremento

Cómo leer los checklists

CA1: resultado funcional. CA2: validaciones y escenarios. Ambos están verificados técnicamente dentro del alcance indicado. CA3: evidencia y documentación existen; referencia del nuevo commit/PR y revisión externa siguen pendientes para todas las fichas. No se oculta ese criterio incompleto.

Los criterios se leen desde docs/planificacion-sprints.json; localProgress expresa el corte actual y los estados históricos del inventario se conservan. Las tallas M/L son estimaciones, no horas medidas ni autoría exclusiva de cada línea.

02. Preparar el entorno de prueba

Primera instalación: seguir la sección de instalación limpia del README. La infraestructura vigente es infra/postgres-v2; infra/postgres es histórica. No copiar .env.example encima de una instalación existente ni restaurar el SQL sobre datos poblados.

V2 ya instalada: actualización ordenada

Desde la raíz, instalar dependencias y arrancar PostgreSQL. Dentro de backend compilar, migrar, preparar permisos y verificar. Detenerse si cualquiera de esos pasos falla.

```
bun install --frozen-lockfile

docker compose --env-file infra/postgres-v2/.env -f

infra/postgres-v2/compose.yaml up -d --wait

cd backend

pnpm install --frozen-lockfile

pnpm run build

node scripts/migrate.mjs up

node scripts/prepare-sprint2.mjs

node scripts/migrate.mjs status

node scripts/verify-provisional-v2.mjs
```

El comando docker compose mostrado en dos líneas se ejecuta como una sola línea. No omitir --env-file ni cambiar la carpeta de infraestructura.

Configuración y procesos

El instalador genera secretos locales y un candidato .env.backend; activate lo coloca en backend/.env conservando la configuración previa. La API local apunta a 127.0.0.1:15433, brotar_db y brotar_app. PostgreSQL escucha 5432 dentro del contenedor. NestJS y Vite corren fuera de Docker.

Publicación: CAMPAIGN_ADMIN_PUBLICATION_ENABLED=true en backend/.env habilita la política provisional, sin conceder ADMIN. No publicar .env. Arrancar pnpm run dev en backend y bun run dev en otra terminal desde la raíz. Abrir http://127.0.0.1:5173 y comprobar http://127.0.0.1:3000/api/health/ready.

Usar las versiones del README: Node 24, Bun 1.4.2 y pnpm 11.19.0. Si el pnpm global difiere, corepack pnpm run lint/typecheck/test/build se ejecutan individualmente. No borrar volúmenes ni usar Factory Reset.

03. Recorrido integral para el ensayo

Preparar tres perspectivas: creador propietario, ADMIN autorizado y visitante sin sesión. Las cuentas de prueba deben ser ficticias. El registro concede rol básico, no CREATOR/ADMIN; no compartir contraseñas ni mostrar .env durante la defensa.

1. Creador: entrar, crear un borrador de modalidad Recompensa y completar información general, historia/impacto y portada pública.

2. Plan y presupuesto: guardar una actividad, un hito, una partida y un riesgo. Guardar ambos bloques. Financiamiento: meta 1000, moneda activa, duración 30 y modelo Flexible.

3. Recompensas: guardar título, descripción, tipo, mínimo 10, cantidad 20 y entrega estimada. Salir y retomar la misma campaña para comprobar persistencia.

4. Revisión final: comprobar resumen y faltantes. Enviar completo; verificar IN_REVIEW y edición bloqueada.

5. ADMIN: abrir la cola y expediente completo. Solicitar cambios con motivo; creador: corregir, guardar y reenviar. Verificar nueva ronda y conservación de la anterior.

6. ADMIN: aprobar y confirmar. Visitante: comprobar que todavía no aparece. ADMIN: publicar y confirmar; visitante: encontrarla con filtros y abrir detalle real.

7. Segundo ensayo: rechazar otra campaña completa con motivo. Tercer borrador desechable: cancelar y luego confirmar descarte. No descartar la campaña principal.

Estado	Qué se puede demostrar
DRAFT	Guardar/continuar; descarte propio confirmado
IN_REVIEW	ADMIN decide; creador no edita; no pública
CHANGES_REQUESTED	Propietario corrige; reenvío crea nueva ronda
APPROVED	Privada; publicación administrativa separada
PUBLISHED	Visible al visitante; no editable aquí
REJECTED	Motivo/historial; no publicación ni reenvío aquí

El PDF oficial define siete áreas; la interfaz mantiene ocho posiciones porque Portada está separada. Donación omite Recompensas. No confundir posición del constructor con número de tarea.

S2-02 | Planificar actividades e hitos

Responsable planificado: Santiago | Esfuerzo: M | Historias: BG-21

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Registrar actividades, responsables y cronograma; relacionar hitos cuando corresponda y validar fechas coherentes. Guardar y recuperar la planificación del borrador propio.

CA2 - VERIFICADO TÉCNICAMENTE. Validar fechas coherentes, responsabilidades, persistencia al salir y volver, campos obligatorios y rechazo de edición por otro usuario.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

El caso CampaignSections valida la propiedad y el estado; validatePlan comprueba responsables, días reales y relación del hito. TypeormSectionsRepository escribe bajo transacción y bloqueo de campaña. El editor conserva el estado de cambios pendientes y recupera las secciones desde la API.

Demostración paso a paso

1. Entrar como creador y abrir Mis borradores. Crear una campaña de ensayo o continuar una propia. Avanzar guardando hasta Plan y presupuesto.

2. Añadir actividad “Plantación de cañahua”, responsable “Ana Muñoz”, inicio 01/11/2026 y fin 10/11/2026. Añadir un hito “Entrega” y asociarlo a la actividad.

3. Pulsar Guardar planificación. Esperar Sección guardada; salir y volver a la misma campaña. Comprobar texto, fechas y vínculo.

4. Quitar una actividad: el editor elimina su vínculo en los hitos y reajusta las posiciones restantes. Guardar y comprobar que no quedan índices inválidos.

Ensayo negativo y resultado esperado

Responsable vacío, fin anterior al inicio o fecha 31/02 deben marcar el campo. Otra cuenta no puede leer ni escribir la planificación privada; el API rechaza el acceso aunque conozca el UUID.

Código y evidencia para defender

backend/src/campaigns/domain/sections.ts:31
Buscar: export function validatePlan

src/features/campaigns/useCampaignSections.ts:9
Buscar: export function useCampaignSections

backend/test/campaign-sections.test.ts; tests/campaign-sections.test.ts; integración campaign-sections-submission.integration.test.ts.

S2-03 | Registrar presupuesto y riesgos

Responsable planificado: Santiago | Esfuerzo: M | Historias: BG-22

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Persistir gastos, partidas de presupuesto y riesgos del proyecto. Validar montos mayores a cero y mostrar totales sin imponer reglas de igualdad con la meta no confirmadas.

CA2 - VERIFICADO TÉCNICAMENTE. Validar montos, longitudes, riesgos y totales; conservar los datos ante errores y comprobar propiedad desde la API.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

El presupuesto y los riesgos se guardan como bloque separado del plan. Se valida la categoría activa y cada importe; los errores usan rutas como items.0.plannedAmount. El total de presentación suma céntimos enteros. No se inventa una obligación de presupuesto igual a meta.

Demostración paso a paso

1. En Plan y presupuesto, añadir una partida y elegir una categoría de gasto activa del catálogo real.

2. Escribir “Semillas”, cantidad 2, precio unitario 5, importe planificado 10 y notas opcionales. Añadir riesgo “Retraso” y mitigación “Proveedor alternativo”.

3. Pulsar Guardar presupuesto y riesgos. Si también se modificó el plan, guardar ambos bloques: el éxito de uno no confirma el otro.

4. Recargar la campaña y comparar partidas, notas, riesgos y total. Probar dos importes 0.10 y 0.20: el total debe presentarse como 0.30.

Ensayo negativo y resultado esperado

Cero, negativos, tres decimales, categoría inactiva, notas con más de 5000 caracteres o carácter NUL no deben reemplazar los datos anteriores. Descripción de campaña con números sí es válida: no es un nombre personal.

Código y evidencia para defender

backend/src/campaigns/domain/sections.ts:45
Buscar: export function validateBudget

src/features/campaigns/sectionsClient.ts:98
Buscar: export function budgetTotal

sprint2-quality.test (frontend/backend); campaign-sections.test; integración con lectura posterior a entradas inválidas.

S2-04 | Configurar el financiamiento

Responsable planificado: Santiago | Esfuerzo: M | Historias: BG-23

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Guardar meta mayor a cero, moneda, duración y modelo de financiamiento admitidos por la base; validar fechas coherentes y explicar los datos requeridos.

CA2 - VERIFICADO TÉCNICAMENTE. Probar meta positiva, moneda y modelo válidos, duración y fechas coherentes, guardado progresivo y respuestas de error por campo.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

Se persisten meta, moneda, duración, modelo y fechas opcionales en la campaña. El backend comprueba que la moneda exista y esté activa. Hay dos modelos del esquema: ALL_OR_NOTHING y FLEXIBLE. Si existen recompensas, se protege el cambio de moneda para no dejar importes incoherentes.

Demostración paso a paso

1. Abrir Financiamiento después de guardar Plan y presupuesto.

2. Introducir meta 1000, moneda BOB si aparece activa, duración 30 y modelo Flexible. Dejar ambas fechas vacías o indicar un par coherente.

3. Pulsar Guardar financiamiento y esperar confirmación. Recargar y comprobar los cuatro datos y las fechas si se introdujeron.

4. Aclarar al docente que aquí se configura financiamiento: no se ejecuta un pago, no se garantiza recaudación y no se confirma cumplimiento de meta.

Ensayo negativo y resultado esperado

Meta 0 o 1.001, duración fraccionaria, moneda inexistente, fin igual/anterior al inicio o solo una fecha deben fallar. 31 de febrero se rechaza, no se convierte silenciosamente en marzo.

Código y evidencia para defender

backend/src/campaigns/domain/sections.ts:68
Buscar: export function validateFunding

backend/src/campaigns/application/sections.ts:68
Buscar: async saveFunding

Validaciones pares frontend/backend y suite PostgreSQL. Las fechas y duración no se fuerzan a equivalencia comercial exacta: esa regla no está confirmada.

S2-05 | Configurar recompensas y preventas

Responsable planificado: Santiago | Esfuerzo: L | Historias: BG-24

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Gestionar tipo, descripción, aporte mínimo, disponibilidad y fecha estimada de recompensas según la modalidad. Omitir el paso cuando no corresponda; conservar cambios sin activar aportes ni pagos.

CA2 - VERIFICADO TÉCNICAMENTE. Probar importes positivos, disponibilidad y fechas, límites de texto y persistencia; omitir recompensas en donación y mantener coherencia al cambiar modalidad.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

Las recompensas incluyen título, descripción, tipo, aporte mínimo, disponibilidad y entrega estimada. El backend verifica modalidad y moneda. Cambiar a donación exige reconocer la existencia de recompensas, pero conserva sus filas; no activa ventas, inventario de aportes ni checkout.

Demostración paso a paso

1. Usar modalidad Recompensa o Preventa. Abrir Recompensas después de Financiamiento.

2. Añadir “Planta de cañahua”, descripción, tipo REWARD o PRESALE según el ensayo, mínimo 10, disponibilidad 20 y fecha 01/01/2027. Guardar.

3. Volver a Modalidad y cambiar a Donación. Si hay recompensas, leer el aviso y confirmar conscientemente el cambio. El asistente omite el paso aplicable.

4. Volver a Recompensa y comprobar que los datos anteriores permanecen. Para enviar esa modalidad hace falta al menos una recompensa completa.

Ensayo negativo y resultado esperado

Mínimo cero/negativo, disponibilidad negativa/fraccionaria, entrega imposible y texto excesivo deben marcar errores. Vacío significa sin límite de disponibilidad; 0 no se confunde con ese vacío.

Código y evidencia para defender

backend/src/campaigns/domain/sections.ts:83
Buscar: export function validateRewards

backend/src/campaigns/application/modality.ts:28
Buscar: export class Modalities

campaign-sections.test.ts y campaign-sections-submission.integration.test.ts: cambio con reconocimiento, conservación y exclusión pública en donación.

S2-10 | Revisar la campaña antes del envío

Responsable planificado: Santiago | Esfuerzo: M | Historias: BG-29

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Mostrar resumen y vista previa de la campaña persistida, indicar campos faltantes y enlazar al paso correspondiente. Cubrir tipo, información general con portada, historia e impacto, plan y presupuesto, financiamiento y recompensas aplicables.

CA2 - VERIFICADO TÉCNICAMENTE. Comparar resumen con datos guardados; señalar exactamente los faltantes, volver al campo correspondiente y impedir envíos incompletos en frontend y backend. Revisar los siete pasos esperados por el PDF sin perder datos existentes.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

CampaignSubmissionPanel consulta datos persistidos y faltantes calculados por missingForSubmission. El resumen no se basa solo en el estado local de inputs. Cada faltante tiene campo, etiqueta y posición del asistente. La portada se presenta como etapa separada sin cambiar el alcance funcional del PDF.

Demostración paso a paso

1. Completar y guardar todos los bloques salvo, por ejemplo, la meta. Abrir Revisión y envío.

2. Leer el resumen y localizar el faltante Meta de financiamiento. Pulsarlo para regresar al paso correcto.

3. Guardar la meta y volver a revisión. Comparar título, resumen, historia, indicadores, portada, plan, presupuesto, financiamiento y recompensas aplicables.

4. Comprobar que los indicadores describen metas esperadas, no resultados alcanzados. La revisión es privada: no equivale a publicar.

Ensayo negativo y resultado esperado

No debe mostrarse completa una campaña con categoría/moneda inactiva, portada privada/inválida o bloque requerido ausente. Un error al consultar no debe convertirse en éxito ni rellenarse con datos simulados.

Código y evidencia para defender

backend/src/campaigns/domain/submission.ts:28
Buscar: export function missingForSubmission

src/features/campaigns/CampaignSubmissionPanel.tsx:25
Buscar: export function CampaignSubmissionPanel

campaign-submission.test.ts, submission-http.test.ts y prueba PostgreSQL integral: respuesta 422 y corrección de faltantes.

S2-11 | Enviar la campaña a revisión

Responsable planificado: Santiago | Esfuerzo: M | Historias: BG-30

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Permitir al creador propietario enviar una campaña completa a pendiente de revisión. Controlar la transición y registrar usuario y fecha en backend; confirmar recepción y evitar envíos duplicados.

CA2 - VERIFICADO TÉCNICAMENTE. Probar campaña incompleta, envío válido, doble envío y acceso ajeno; comprobar el rol creador y la transición registrada en la API.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

El envío reutiliza la sesión existente y exige REGISTERED_USER y CREATOR. La API comprueba propiedad, estado editable y completitud; el repositorio vuelve a comprobar bajo bloqueo. La nueva ronda, el estado IN_REVIEW y el historial se registran de manera transaccional.

Demostración paso a paso

1. Iniciar sesión con una cuenta de ensayo que tenga CREATOR realmente almacenado. Elegir “creador” en el registro no concede el rol.

2. En Revisión y envío, resolver todos los faltantes y guardar. Pulsar Enviar a revisión una vez y esperar el resultado.

3. Abrir Mis proyectos y comprobar En revisión. Intentar volver a editar: la interfaz y la API deben bloquear la escritura.

4. Como administrador, abrir la cola y verificar que aparece esa campaña; todavía no debe encontrarse en Explorar sin sesión.

Ensayo negativo y resultado esperado

Sin sesión: 401; sin CREATOR: 403; campaña incompleta: 422; otro propietario: acceso denegado/no disponible. Dos envíos concurrentes no crean dos rondas: la prueba obtiene un éxito y un 409.

Código y evidencia para defender

backend/src/campaigns/application/submission.ts:15
Buscar: export class CampaignSubmission

backend/src/campaigns/infrastructure/typeorm-submission.repository.ts:9
Buscar: export class TypeormSubmissionRepository

submission-http.test.ts, campaign-submission.test.ts y escenarios de concurrencia/persistencia del ensayo PostgreSQL integral.

S2-12 | Decidir sobre campañas recibidas

Responsable planificado: Ricardo | Esfuerzo: L | Historias: BG-42

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Permitir al administrador consultar campañas pendientes y toda la información enviada; aprobar, rechazar con motivo o solicitar cambios con comentarios al creador. Registrar la decisión y su responsable.

CA2 - VERIFICADO TÉCNICAMENTE. Probar las tres decisiones, motivos y comentarios requeridos, historial y denegación en la API a usuarios sin permiso administrativo; aprobación no implica publicación.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

ADMIN consulta el expediente completo y decide aprobar, solicitar cambios o rechazar. Se registra actor autenticado, fecha, motivo y observaciones; no se acepta que el cuerpo elija el revisor. Una ronda esperada impide decidir sobre una revisión que ya cambió.

Demostración paso a paso

1. Iniciar sesión como ADMIN y abrir Mi cuenta > Opciones para iniciativas y permisos > Revisión de campañas.

2. Abrir el detalle de una campaña IN_REVIEW. Revisar todos los bloques, portada, faltantes, rondas y estado: no solo nombre y meta.

3. Elegir Solicitar cambios, escribir un motivo entendible y una observación por sección si corresponde; confirmar. Comprobar CHANGES_REQUESTED.

4. En otro ensayo, aprobar y comprobar APPROVED privada. Para rechazo usar una segunda campaña enviada y un motivo; comprobar REJECTED y su historial.

Ensayo negativo y resultado esperado

Solicitud de cambios o rechazo sin motivo, NUL, más de 5000 caracteres, sección inventada, campos extra y ronda obsoleta no deben escribir. Si la respuesta es incierta, actualizar antes de intentar decidir de nuevo.

Código y evidencia para defender

backend/src/campaigns/domain/administrative-review.ts:12
Buscar: export function validateDecision

backend/src/campaigns/infrastructure/typeorm-administrative.repository.ts:70
Buscar: decide(actorId

administrative-review.test.ts; administrative-review-http.test.ts; administrative-campaigns.test.ts; ensayo completo PostgreSQL.

S2-13 | Corregir observaciones y reenviar

Responsable planificado: Santiago | Esfuerzo: M | Historias: BG-31

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Mostrar al creador propietario los cambios solicitados; permitir corregir las secciones habilitadas y reenviar la campaña sin perder datos ni historial de decisiones.

CA2 - VERIFICADO TÉCNICAMENTE. Probar lectura de observaciones, corrección persistente, reenvío y restricciones de estado y propiedad; no admitir modificaciones ajenas.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

Solo CHANGES_REQUESTED permite la corrección del propietario. Las observaciones de la última ronda se muestran en el constructor. Guardar no borra el historial; reenviar crea otra ronda pendiente y vuelve a bloquear la edición.

Demostración paso a paso

1. Después de una solicitud de cambios real, salir de ADMIN e iniciar sesión como creador propietario.

2. En Mis proyectos, abrir Corregir campaña. Consultar las observaciones de la revisión y ubicar el bloque afectado.

3. Corregir, por ejemplo, la explicación de beneficiarios. Guardar ese bloque; volver a Revisión y envío y comprobar que no quedan faltantes.

4. Reenviar. Como ADMIN, actualizar el expediente: debe existir una ronda nueva PENDING y conservarse la decisión y observaciones anteriores.

Ensayo negativo y resultado esperado

Otra persona no puede corregir el borrador. IN_REVIEW, APPROVED, PUBLISHED y REJECTED no permiten edición/reenvío en este incremento. Rechazado no significa “cambios solicitados”.

Código y evidencia para defender

backend/src/campaigns/domain/draft.ts:62
Buscar: export function allowsBuilderEditing

src/features/campaigns/CampaignSubmissionPanel.tsx:25
Buscar: export function CampaignSubmissionPanel

Prueba PostgreSQL: varias rondas, observaciones, reenvío y conflictos con la ronda anterior; pruebas de dominio de estados.

S2-14 | Publicar y controlar estados de campaña

Responsable planificado: Ricardo | Esfuerzo: L | Historias: BG-32

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Controlar y registrar en backend borrador, pendiente de revisión, cambios solicitados, aprobada, publicada y rechazada. Separar aprobación de publicación y hacer visible solo lo publicado; conservar motivo de rechazo e historial.

CA2 - VERIFICADO TÉCNICAMENTE. Probar transiciones válidas y rechazadas, permisos de cada actor e historial; excluir de la API pública las campañas aprobadas pero no publicadas y las demás privadas.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

Aprobar y publicar son operaciones distintas. D05 autoriza provisionalmente solo ADMIN a publicar con la bandera habilitada. La publicación exige aprobación vigente, confirmación expresa, ronda correcta y campaña completa; registra fecha/actor y deja el recurso visible al visitante.

Demostración paso a paso

1. Como ADMIN, aprobar la campaña enviada y confirmar. Leer el aviso de que APPROVED sigue privada.

2. Sin sesión, buscarla en Explorar: aún no debe aparecer. Volver al expediente administrativo actualizado.

3. Pulsar Publicar campaña y confirmar. Comprobar PUBLISHED y el historial de publicación.

4. Sin sesión, buscar y abrir el proyecto publicado. Comparar con los datos que guardó el creador. No editar el estado con SQL para simular esta demostración.

Ensayo negativo y resultado esperado

Sin ADMIN, sin bandera habilitada, sin confirmación, en DRAFT/IN_REVIEW/REJECTED o con ronda obsoleta se deniega. La bandera no concede roles. Publicar dos veces no duplica el historial.

Código y evidencia para defender

backend/src/campaigns/application/administrative-review.ts:21
Buscar: publish(actorId

backend/src/campaigns/domain/lifecycle.ts:2
Buscar: export const PHASE3_STATUSES

D05 en ADR-005; migración CampaignRejected; pruebas HTTP, dominio, migración aislada y PostgreSQL con publicación concurrente.

S2-15 | Gestionar proyectos propios y descartar borradores

Responsable planificado: Ricardo | Esfuerzo: L | Historias: BG-13, BG-14

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Listar campañas propias con detalle y acciones según estado; continuar, consultar y actualizar el borrador. Permitir eliminar o descartar únicamente borradores propios con confirmación y control en backend.

CA2 - VERIFICADO TÉCNICAMENTE. Probar lista vacía, detalle propio, continuación, acciones por estado y denegación a otro usuario. El descarte se verifica como criterio adicional de BG-14.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

CA4 - VERIFICADO TÉCNICAMENTE. Probar descarte confirmado de borrador propio, rechazo de otro propietario y rechazo de estados no borrador; recuperar la lista sin datos descartados.

Implementación y motivo

Projects y TypeormProjectRepository listan solo proyectos propios y permiten abrir un proyecto concreto. Las acciones dependen del estado. El descarte es lógico, exige confirmación y solo afecta DRAFT propio; mantiene relaciones e historial.

Demostración paso a paso

1. Como creador, abrir Mis proyectos. Identificar título, estado y actualización de cada campaña propia.

2. Abrir el detalle y pulsar Continuar borrador o Corregir campaña según el estado. Comprobar que abre esa campaña y no otra.

3. Crear otro borrador desechable. Pedir Descartar, cancelar y comprobar que sigue visible. Volver a pedirlo y confirmar.

4. Comprobar que el descartado sale de la lista habitual y no se publica. No usar la campaña principal de la presentación para el ensayo de descarte.

Ensayo negativo y resultado esperado

Una cuenta distinta no debe consultar/descartar esa campaña; tampoco se descartan campañas enviadas, aprobadas o publicadas. Una carrera entre envío y descarte debe abortar sin un historial incoherente.

Código y evidencia para defender

backend/src/campaigns/application/projects.ts:31
Buscar: async discard

src/features/campaigns/ProjectsPage.tsx:26
Buscar: export function ProjectsPage

projects.test.ts, projects.integration.test.ts y pruebas de frontend de listas, confirmación, estados y errores.

S2-16 | Conectar catálogo y detalle público con campañas reales

Responsable planificado: Alison | Esfuerzo: L | Historias: BG-01, BG-05, BG-06, BG-07, BG-08, BG-12

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Sustituir campañas simuladas por la API pública. Mostrar solo publicadas con imagen, nombre, categoría, avance, meta y creador; buscar por texto y filtrar por categoría, ubicación y tipo. Incluir historia, problema, solución, beneficiarios, impacto y recompensas en el detalle.

CA2 - VERIFICADO TÉCNICAMENTE. Probar filtros, resultados vacíos y errores; comprobar contenido real y exclusión de campañas privadas. El avance debe proceder de datos reales, sin recaudaciones ficticias.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

El catálogo y detalle públicos leen la API real; el repositorio filtra PUBLISHED, no descartadas y portada pública válida. El contrato excluye contenido administrativo. El avance procede de la vista de financiamiento, no de cifras de demostración.

Demostración paso a paso

1. Cerrar sesión o usar una ventana privada. Abrir Explorar; debe verse la campaña publicada del ensayo anterior.

2. Buscar por parte de su nombre y combinar categoría, ubicación y modalidad. Los filtros se intersectan, no se suman indiscriminadamente.

3. Abrir la tarjeta: comprobar portada, creador, problema, solución, beneficiarios, impacto esperado, meta/avance y recompensas aplicables.

4. Buscar un texto inexistente y observar el vacío explicado. Si no hay publicaciones, no deben aparecer automáticamente las antiguas campañas mock.

Ensayo negativo y resultado esperado

DRAFT, IN_REVIEW, CHANGES_REQUESTED, APPROVED y REJECTED no se muestran. Un slug privado/inexistente devuelve no disponible. Fallo de red o JSON malformado no se transforma en una campaña inventada.

Código y evidencia para defender

backend/src/campaigns/infrastructure/typeorm-public-campaign.repository.ts:7
Buscar: export class TypeormPublicCampaignRepository

src/features/public/campaignsApi.ts:23
Buscar: export function mapPublicCampaign

public-campaigns.test.ts de ambas aplicaciones y public-campaigns.integration.test.ts. Favoritos, aportes y pagos quedan fuera del alcance actual.

S2-17 | Preparar acceso y cola administrativa de campañas

Responsable planificado: Ricardo | Esfuerzo: M | Historias: BG-41

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Preparar navegación y acceso administrativo para listar campañas pendientes y abrir su detalle. Reutilizar la revisión de S2-12; no construir un panel completo con estadísticas ni colas KYC/KYB.

CA2 - VERIFICADO TÉCNICAMENTE. Probar acceso permitido y denegado, lista vacía, carga y error. Diferenciar esta tarea de S2-12: navegación y cola frente a decisiones y registro de revisión.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

La cola administrativa reutiliza la lista/detalle y el panel de decisión; distingue navegación de S2-12. Incluye campañas enviadas y aprobadas pendientes de publicación. La autorización se comprueba en servidor, además del acceso condicional de la interfaz.

Demostración paso a paso

1. Entrar con ADMIN; desde Mi cuenta abrir Revisión de campañas. Consultar la lista y buscar la campaña de ensayo.

2. Filtrar por estado, abrir el detalle y comprobar que mantiene el proyecto seleccionado. Usar Volver para regresar a la cola.

3. Después de aprobar, volver a la cola: APPROVED debe seguir disponible para publicar; no se pierde entre revisión y publicación.

4. Con una cuenta básica intentar abrir /administracion/campanas: debe verse denegación, no una cola con datos ajenos.

Ensayo negativo y resultado esperado

Sin sesión se solicita iniciar sesión; un usuario común no recibe datos administrativos. Lista vacía, carga y error se distinguen. La cola no incorpora estadísticas financieras ni revisión KYC/KYB de Sprint 3.

Código y evidencia para defender

backend/src/campaigns/infrastructure/projects.controller.ts:30
Buscar: export class CampaignReviewQueueController

src/features/campaigns/useProjects.ts:5
Buscar: export function useProjects

projects.integration.test.ts, pruebas HTTP de proyectos y pruebas frontend de cola/accesos. Provisión definitiva de ADMIN sigue pendiente en D06.

S2-21 | Verificar y documentar el segundo incremento

Responsable planificado: Ricardo | Esfuerzo: L | Historias: BG-04, BG-53, BG-60, BG-61, BG-62, BG-63

Qué debía cumplirse y estado del checklist

CA1 - VERIFICADO TÉCNICAMENTE. Probar y documentar el recorrido real de creador, borrador por pasos, continuidad, descarte, revisión administrativa, correcciones, aprobación, publicación y exploración pública. Actualizar README, contratos y evidencias sin cambiar la arquitectura.

CA2 - VERIFICADO TÉCNICAMENTE. Validar campos obligatorios, longitudes, montos positivos, fechas coherentes, carga/éxito/error y solicitudes duplicadas; comprobar autenticación existente, rol creador, propiedad, permiso administrativo, móvil y teclado. Checkout, pagos y KYC/KYB quedan fuera de estas pruebas de aceptación.

CA3 - PARCIAL: entrega y aceptación externas pendientes. Adjuntar evidencia reproducible, pruebas y referencia de commit/PR; actualizar documentación y obtener revisión del alcance. No confundir cierre técnico con aceptación externa.

Implementación y motivo

Se integraron los trece compromisos y se ejecutaron validaciones, lint, tipos, compilaciones y pruebas reales de PostgreSQL. Se documentaron instalación, permisos, D05 y recorridos. Esta revisión añade dos guías y casos de Unicode, precisión decimal, fechas y NUL.

Demostración paso a paso

1. Seguir la preparación de esta guía o el README vigente. Comprobar Docker saludable y /api/health/ready antes de evaluar funcionalidades.

2. Ejecutar bun run check; en backend las comprobaciones con pnpm 11.19.0; ejecutar la integración con escritura de fixtures autorizada.

3. Guardar la salida resumida: 130 frontend, 128 backend y 18 integración, todos aprobados. Estos 276 casos incluyen regresiones del Sprint 1.

4. Demostrar creador > administrador > visitante con datos ficticios. Anotar qué se observó, resultado esperado, resultado real y responsable de la revisión.

Ensayo negativo y resultado esperado

No marcar aceptación externa por tener tests verdes. Falta publicar el corte con referencia de commit/PR cuando se autorice y obtener revisión de líderes. No declarar correo, legal, KYC/KYB, pagos ni provisión definitiva de roles resueltos.

Código y evidencia para defender

tests/sprint2-quality.test.tsx:18
Buscar: test('S2 calidad

backend/integration/campaign-sections-submission.integration.test.ts:20
Buscar: test('S2 completo PostgreSQL

Los tests de integración limpian solo sus cuentas, campañas y catálogos temporales. No se borraron volúmenes ni datos previos. El ensayo UI del corte anterior se conserva separado de las pruebas automáticas actuales.

04. Repetir las pruebas y registrar evidencia

Conjunto	Aprobadas	Fallidas / omitidas
Frontend	130	0 / 0
Backend sin base	128	0 / 0
PostgreSQL real	18	0 / 0
Total	276	0 / 0

En raíz: bun run check. Backend: pnpm run check con la versión declarada. En el equipo de la revisión, el pnpm global era distinto: se ejecutaron individualmente corepack pnpm run lint, typecheck, test y build. La suite de tests compila dist-test antes de la integración.

```
# Desde backend, con V2 configurada y permisos preparados
$env:ALLOW_DB_TEST_WRITES='true'
$env:DB_TEST_RESTART='false'
node --env-file=../infra/postgres-v2/.env.backend --test
--test-concurrency=1 dist-test/integration/*.test.js
Remove-Item Env:\ALLOW_DB_TEST_WRITES
Remove-Item Env:\DB_TEST_RESTART
```

La invocación node partida en dos líneas se ejecuta como una línea. El README usa try/finally para retirar las variables aunque falle el ensayo. Solo ejecutar escritura de fixtures en la base local de ensayo, nunca en producción.

Qué prueban realmente

La integración usa HTTP, roles y PostgreSQL reales: propiedad, validación, rollback, lectura posterior, reinicio de API, sesión, rondas, solicitudes concurrentes, aprobación privada, publicación y catálogo. La migración se ensaya también sobre una base temporal restaurada desde SQL oficial. En este corte DB_TEST_RESTART=false: no se repitió el reinicio del contenedor.

Nuevas regresiones de esta revisión

Ñ, acentos y espacios exactos en hash/login; 128 puntos de código; nombres personales con números rechazados; importes válidos altos; total 0.10 + 0.20; febrero imposible; límites 5000; NUL rechazado antes de PostgreSQL; ronda administrativa inválida; carga inicial accesible tras extraer el hook.

Limitaciones de la evidencia

La interfaz real fue ensayada en el corte previo (capturas en docs/evidencia-sprint-2). La comprobación adicional por navegador de esta revisión quedó bloqueada por el acceso de la herramienta: no se declara repetida. El render SSR comprueba contenido inicial, no sustituye una prueba interactiva completa del hook.

05. Defensa, contingencias y cierre

Errores de entorno: cómo explicar y actuar

Si /health/ready falla, revisar Docker saludable, DATABASE_ENABLED, puerto 15433 y configuración activada sin imprimir secretos. Si falta app_user/user_profile, detenerse: la migración no instala una línea base ausente; seguir instalación limpia. No restaurar sobre una base poblada para “arreglarla”.

403 o botones ausentes

Revisar sesión y roles guardados. Perfil elegido en registro, estado PENDING_VERIFICATION y verificación de identidad son conceptos distintos. La cuenta puede tener acceso básico sin recibir CREATOR o ADMIN. No eliminar guards para facilitar la presentación.

409, 422 y respuesta incierta

409: estado/ronda cambió; actualizar antes de repetir. 422: revisar los faltantes y guardar los bloques indicados. Si se perdió la respuesta del guardado, los datos del formulario se conservan, pero no afirmar que el servidor no escribió: recuperar el estado antes de otra decisión administrativa.

Preguntas frecuentes de defensa

¿Por qué aprobar no publica? Para separar evaluación y exposición pública, conforme a D05 provisional. ¿Por qué sigue vacía Explorar? No hay PUBLISHED con portada pública válida o el filtro excluye resultados. ¿Dónde se valida? Frontend orienta; DTO/dominio/caso de uso y transacción protegen la escritura. ¿Se cerró todo Brotar? No: solo el alcance técnico de estas trece tareas.

Pendientes explícitos

Entrega del nuevo corte con referencia de commit/PR y aceptación de líderes; provisión definitiva de roles D06; política final de publicación/negocio del cliente; correo real, textos legales y verificación cuando corresponda. Estas decisiones no se resuelven falsamente mediante SQL ni con pruebas verdes.

Fuentes y material de apoyo

Brotar_Tarea_Fase3_Modulo_Campanas.pdf, secciones 4-10 y 13; docs/planificacion-sprints.json (trece tareas); README, secciones 3, 12, 14 y 15; docs/cierre-tecnico-sprint-2-2026-10-05.md; ADR-005; código y suites citados en cada ficha. El segundo PDF explica los principios de diseño que sustentan la implementación.

Ficha de ensayo sugerida: tarea / cuenta y rol ficticios / pasos / resultado esperado / resultado observado / prueba o captura / fecha / revisor. Marcar “no ejecutado” cuando no se comprobó, no “aprobado por suposición”.